

Clase 099 — Server-Side Request Forgery (SSRF)

Parte: 4 — Seguridad de aplicaciones web · Fuente: Real-World Bug Hunting (Yaworski) / OWASP Top 10 A10 🕒 Duración estimada: 120 min · Nivel: Avanzado

🎯 Objetivo

Explotar el **SSRF (falsificación de petición del lado servidor)**: hacer que el servidor realice peticiones a destinos que el atacante controla, incluyendo servicios internos y endpoints de metadata cloud. Es una de las vulnerabilidades de mayor impacto en entornos modernos y una categoría propia del OWASP Top 10.

⚠️ **Ética:** solo en labs propios/autorizados (PortSwigger, Juice Shop). Alcanzar redes internas ajenas es un delito.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Identificar** funciones que hacen que el servidor pida URLs (webhooks, importadores, previews).
- 2. **Explotar** SSRF para alcanzar `localhost` y la red interna.
- 3. **Extraer** credenciales del endpoint de metadata cloud.
- 4. **Aplicar** SSRF ciega con detección out-of-band.
- 5. **Evadir** filtros de URL débiles y recomendar defensas robustas.

📖 Temas

#	Tema	Por qué importa
1	Qué es SSRF y su superficie	Base del ataque
2	SSRF a servicios internos	Pivote hacia la red interna
3	Metadata cloud (169.254.169.254)	Robo de credenciales
4	SSRF ciega y OOB	Detección sin respuesta directa
5	Bypass de filtros de URL	Realidad de las defensas
6	Esquemas alternativos (file://, gopher://)	Amplían el impacto
7	Defensa: allowlist, sin redirecciones	Cierre del fallo

📖 Definiciones y características

- **SSRF:** el servidor hace una petición a una URL controlada por el atacante. Característica: usa la posición de red del servidor.

- **Metadata endpoint:** servicio cloud interno que entrega credenciales temporales (AWS/GCP/Azure). Característica: solo accesible desde la instancia, ideal objetivo de SSRF.
- **SSRF ciega:** no se ve la respuesta; se confirma por interacción OOB. Característica: requiere Collaborator/servidor propio.
- **Bypass de filtro:** técnicas para saltar allow/blocklists (IP decimal, DNS rebinding, redirecciones). Característica: los filtros por string son frágiles.
- **Esquema de URL:** `http`, `file`, `gopher`, `dict`. Característica: esquemas exóticos amplían lo que se puede hacer.
- **DNS rebinding:** cambiar la resolución DNS tras la validación. Característica: evade filtros basados en resolver una vez.

Herramientas y preparación

- **PortSwigger labs** de SSRF y **Juice Shop**.
- **Burp Collaborator** o servidor propio para OOB.
- Conocer los endpoints de metadata de cada nube (documentación oficial).

Laboratorio guiado

 Solo en labs propios/autorizados.

1. Localiza una función que reciba una URL (importar imagen, webhook, comprobar stock).
2. Cambia la URL a `http://localhost/admin` y observa si el servidor la alcanza.
3. Escanea puertos internos cambiando el puerto en la URL y midiendo respuestas/tiempos.
4. En el lab cloud de PortSwigger, apunta al endpoint de metadata:

```
http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

5. Extrae el rol y luego las **credenciales temporales** de ese rol.
6. Para SSRF ciega, apunta la URL a tu Collaborator y confirma la interacción DNS/HTTP.
7. Evade un filtro que bloquea `localhost` usando `127.1`, `:::1` o IP en formato decimal.

Ejercicios

1. Enumera 5 features típicas que introducen SSRF.
2. Explica por qué el endpoint de metadata es tan crítico.
3. Evade un blocklist de `127.0.0.1` de tres formas distintas.
4. Usa una redirección abierta para saltar un allowlist de dominios.
5. Diferencia SSRF de CSRF con claridad.
6. Diseña una defensa: allowlist de destinos + bloqueo de IPs privadas + no seguir redirecciones.

Reto verificable

Resuelve un lab de SSRF de PortSwigger que exija **acceder al endpoint de metadata** y usa las credenciales obtenidas para completar el objetivo del lab. **Criterio de aceptación:** el lab queda resuelto, entregas la URL

de SSRF, las credenciales/dato extraído y explicas la defensa (allowlist, bloqueo de rangos internos) que lo evitaría.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El servidor no sigue la URL	La feature valida el destino; busca otra función
Bloqueado <code>localhost</code>	Usa <code>127.1</code> , <code>[::1]</code> , decimal o DNS que resuelva a interno
Sin respuesta visible	SSRF ciega; confirma con OOB
Redirección no ayuda	La app no sigue redirecciones; prueba otro vector
Metadata devuelve 401	IMDSv2 requiere token; ajusta la técnica

? Preguntas frecuentes

? **¿Por qué SSRF es tan grave en cloud?** Porque el endpoint de metadata entrega credenciales de la instancia; con ellas, el atacante puede pivotar a toda la cuenta cloud.

? **¿IMDSv2 resuelve el SSRF?** Lo mitiga exigiendo un token PUT previo, más difícil de lograr vía SSRF simple, pero no elimina todos los vectores.

? **¿Basta con un blocklist de IPs?** No. Los blocklists se evaden fácil. Usa allowlists de destinos y bloquea rangos privados a nivel de red.

🔗 Referencias

- Yaworski, *Real-World Bug Hunting*, cap. de SSRF.
- OWASP SSRF: https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
- OWASP SSRF Prevention Cheat Sheet.
- PortSwigger SSRF: <https://portswigger.net/web-security/ssrf>

➡ Siguiente clase

Clase 100 - XML External Entities (XXE)